Digital Signal Processing – Assignment 1
Fourier Transform: Voice Analysis, Enhancement and Aural Exciter

William Chambers (GUID: 2906463C)
Saad Aziz (GUID: 2917444A)

University of Glasgow, School of Engineering

October 18, 2026

1 Recordings

The voice is Will's, speaking one English sentence: "*Saad and Will are electrowizards.*" Both recordings are uncompressed WAV, 48 kHz, 24-bit PCM (stored in 32-bit integers), mono, 4 s long (192 000 samples), not clipped, at distances of 5 cm and 1 m. The Nyquist frequency is 24 kHz, so the full spectrum up to 20 kHz is available. The peak amplitudes are 0.0753 (−22.5 dBFS) at 5 cm and 0.0060 (−44.5 dBFS) at 1 m, so neither recording is clipped; the 1 m recording is 22 dB quieter at its peak.

2 Task 1: Time and Frequency Analysis

2.1 Time and frequency domain plots

The samples are read with `wavfile.read` and normalised by dividing by 2^{31}, the full scale of the 32-bit container holding the 24-bit data, giving amplitudes in $[-1, 1)$. The time axis is built with `linspace` from 0 s to N/f_s. The spectrum is $X = \frac{1}{N}\,\mathrm{FFT}(x)$, with the $1/N$ scaling making the magnitude independent of recording length, and is shown as $20\log_{10}|X|$ in dBFS. Only the first $N/2$ bins are plotted, from DC up to the Nyquist frequency, with bin spacing f_s/N, because the spectrum of a real signal is mirrored. The frequency axis is logarithmic.

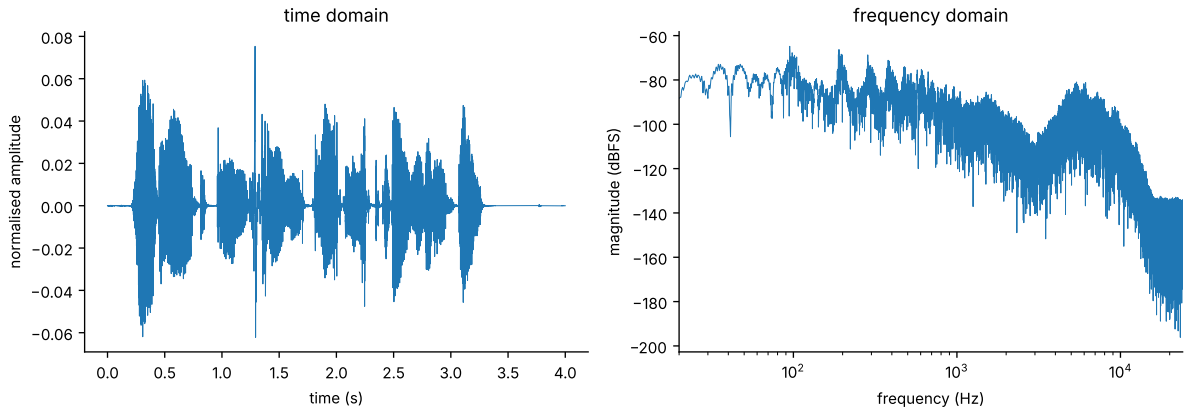


Figure 1: Voice recorded at 5 cm: (left) time domain, (right) frequency domain.

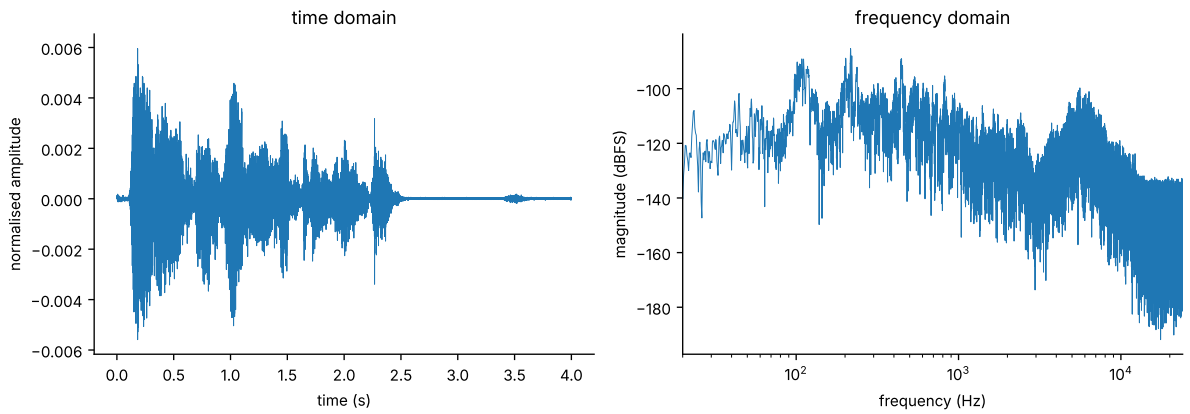


Figure 2: Voice recorded at 1 m: (left) time domain, (right) frequency domain.

2.2 Vowel fundamentals

In both recordings the vowel energy appears as a series of peaks, the first at 93 Hz to 117 Hz, with the following peaks spaced at the same interval. At 5 cm, for example, the peaks in one vowel lie at 104, 208, 312 and 416 Hz; in the another at 95, 190, 285 and 380 Hz. This equal spacing is the harmonic series of a voiced vowel, so the lowest peak of each series was taken as the fundamental f_0. A fundamental of about 100 Hz is consistent with an adult male speaker. The pitch drifts through the sentence, from 104 Hz to 93 Hz at 5 cm, and starts higher in the 1 m take (117 Hz, falling to about 100 Hz), because the two recordings are separate takes.

Table 1: Visually identified vowel fundamentals.

Vowel	f_0 at 5 cm (Hz)	f_0 at 1 m (Hz)	Justification
Vowel 1	≈ 104	≈ 118	Harmonics at $f_0, 2f_0, 3f_0, \ldots$
Vowel 2	≈ 95	≈ 100	Harmonics at $f_0, 2f_0, 3f_0, \ldots$
Vowel 3	≈ 93	≈ 95	Harmonics at $f_0, 2f_0, 3f_0, \ldots$

2.3 Consonant range

Above about 2 kHz the harmonic series fades and a broad, noise-like hump appears with its peak at 5 kHz to 6 kHz. The absence of harmonic structure and the broadband shape are characteristic of consonants such as /s/ and /z/ in the sentence. At 5 cm the hump reaches about −82 dBFS at 6 kHz and the level stays above the noise floor up to about 12 kHz, after which it falls steeply to the floor at about 16 kHz. The consonant range is therefore marked from 2 kHz to 12 kHz. At 1 m the hump is lower in level (about −107 dBFS at 5 kHz) and drops into the noise floor sooner, reaching it at about 14 kHz, so the range is marked 2 kHz to 10 kHz.

2.4 Noise region

Above 16 kHz (14 kHz at 1 m) both spectra are flat at about −143 dBFS (power level smoothed over 50 Hz) with no harmonic structure. The level is the same at both distances, so it does not come from the voice and is attributed to electronic noise of the recording chain. The silent part at the end of each recording is consistent with this: it has an RMS level of −96.6 dBFS at 1 m, flat across the spectrum at about −137 dBFS per bin, and −90.7 dBFS at 5 cm, with the extra energy at low frequencies (about −122 dBFS per bin between 60 Hz to 300 Hz against −135 dBFS above 5 kHz). The region 16 kHz to 24 kHz is marked as noise only for both recordings.

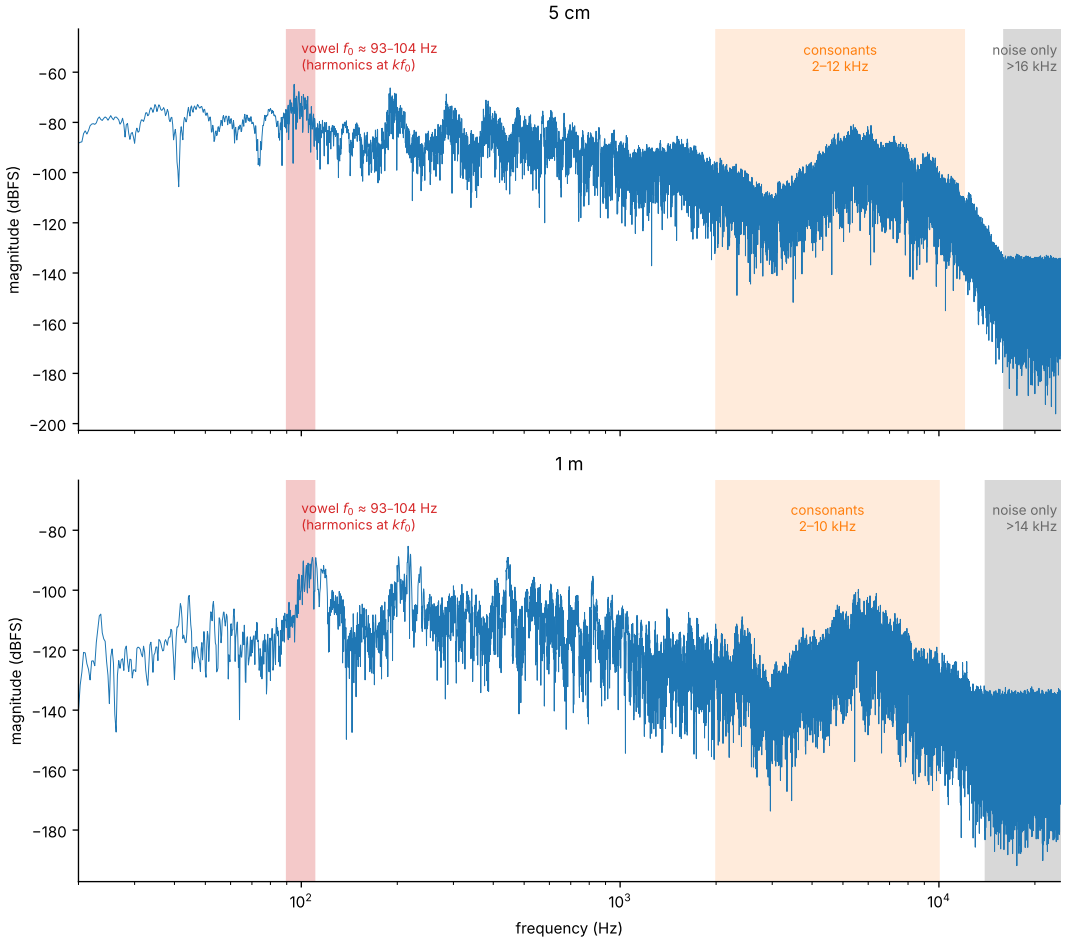


Figure 3: Annotated spectra: vowel fundamentals, consonant range and noise-only region, for 5 cm (top) and 1 m (bottom).

3 Task 2: Speech Enhancement

3.1 Method

The recording is transformed with the FFT, every bin is multiplied by a gain array and the result is transformed back with the IFFT. The gain array has the same length N as the signal. A band is defined by `gain_ctrl(low, high, gain_dB, ...)`: a trapezoidal profile in dB is built over the positive frequencies with `np.interp`, converted to linear gain with $10^{\mathrm{dB}/20}$, multiplied into the array, and mirrored onto the negative-frequency bins so that the IFFT output is real. The parameter $q \in [0, 1]$ sets the fraction of the band width over which each edge ramps ($q = 0$ ramps over one bin, a rectangle). Hard edges produced audible ringing (Gibbs phenomenon), so the band edges are interpolated, with $q = 0.1$ for broad bands and $q = 0.25$–0.3 for the narrower bands. Bands that start at DC or end at Nyquist have their ramp placed outside the spectrum so the ends receive the full gain, which fixed a gain curve that rose at both ends of the spectrum. Several bands can be stacked on one array by calling `gain_ctrl` repeatedly. After the IFFT only the real part is kept (`np.real`); the imaginary part is numerically close to zero because of the mirrored gain array. An identity test, with a gain array of ones, confirmed that the output equals the input (`np.allclose`), so the pipeline itself introduces no change. The result is scaled by $2^{31} - 1$ and saved as 32-bit integer WAV, the same format as the input.

3.2 Noise to be removed

5 cm: low-frequency rumble below about 80 Hz, which lies below the fundamental of the voice, and hiss above about 12 kHz, where no voice energy remains. Above 16 kHz only the flat electronic floor of about −143 dBFS is present. The spectrum between 20 Hz to 100 Hz is at −70 dBFS to −80 dBFS, as high as the voice harmonics themselves, whereas the 1 m recording has only about −107 dBFS at 50 Hz.

1 m: room rumble below about 85 Hz, a narrow spike near 500 Hz (480 Hz to 505 Hz), possibly the 10th harmonic of mains hum, a room-resonance bump around 2.2 kHz to 2.8 kHz that is higher than in the 5 cm recording, and hiss above 13 kHz, where the consonants have already ended. The recording at 1 m also contains room reverberation, which cannot be removed with a frequency-domain gain, so an audible difference to the 5 cm recording remains.

3.3 Bass loss with distance and pop sounds

5 cm: close to the microphone the low frequencies are already clear, so the bass boost is small, and the main risk is pop sounds. A pop was identified between 1.28 s and 1.30 s (measured in a DAW), lasting about 50 ms. In the 1.28 s to 1.30 s window the RMS level is −29.8 dBFS, 15 dB above the −44.9 dBFS of the preceding 80 ms; the same window in the 1 m recording shows no such rise (−60.5 dBFS against −61.7 dBFS). Most of its energy lies below 150 Hz, but it extends throughout the spectrum and so cannot be removed completely with a gain curve. A low cut from DC to 80 Hz, below the fundamental of about 95 Hz, removes rumble and the largest part of the pop energy, and a modest boost of 4 dB between 95 Hz and 400 Hz, covering the first harmonics, adds depth. Brightness is added in the noisy consonant region (3 kHz to 10 kHz) for clarity, and hiss is cut above 12.2 kHz.

1 m: the 1 m recording is about 21 dB quieter than the 5 cm recording at 100 Hz and 17 dB quieter at 1 kHz, and 35 dB quieter at 50 Hz (smoothed spectra). Bass is lost with distance and the fundamental measured higher than at 5 cm (Table 1), so a larger boost of 6 dB is applied between 95 Hz and 180 Hz to restore depth, with a low cut below 85 Hz to remove ambient rumble. The overall level of the 1 m recording is raised by 21 dB so that its peaks match the 5 cm recording (raw peaks −44.5 dBFS and −22.5 dBFS); the enhanced peaks are 0.122 and 0.147 of full scale. This also lifts the noise floor, which is audible once the voice ends (RMS −73 dBFS in the enhanced 1 m recording, from −96.6 dBFS before enhancement, against −94 dBFS for the enhanced 5 cm recording), and is the reason for the strong high cut and the notch at 500 Hz. The band-wise gains in Table 2 are applied on top of the 21 dB level match.

Table 2: Gain adjustments applied (final settings). For 1 m a general gain of 21 dB over the whole spectrum is applied first and the bands below are added to it.

Recording	Band (Hz)	Gain (dB)	q	Purpose
5 cm	0–80	−16	0.1	Low cut: rumble and pop energy
5 cm	95–400	+4	0.1	Bass boost: first harmonics
5 cm	3000–10000	+6	0.1	Brightness: consonant region
5 cm	12200–Nyq	−12	0.1	High cut: hiss
1 m	0–Nyq	+21	0	Volume match to 5 cm peaks
1 m	0–85	−12	0.3	Low cut: room rumble
1 m	95–180	+6	0.3	Bass boost: restore depth
1 m	480–505	−12	0.25	Notch: 500 Hz spike
1 m	2200–2800	−4	0.25	Room resonance
1 m	3000–10000	+7	0.1	Brightness
1 m	13000–Nyq	−30	0.1	High cut: hiss

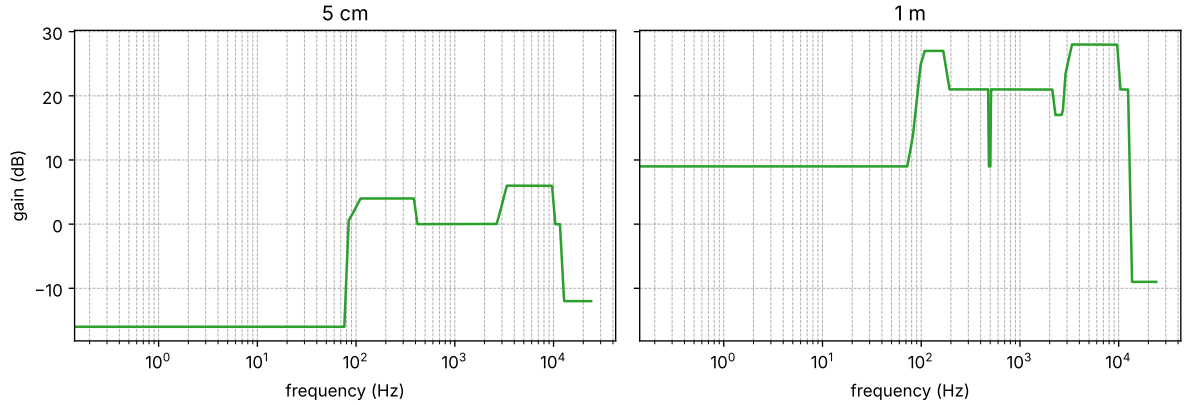


Figure 4: Gain control curves applied to each recording (net gain per frequency, including the 21 dB level match at 1 m).

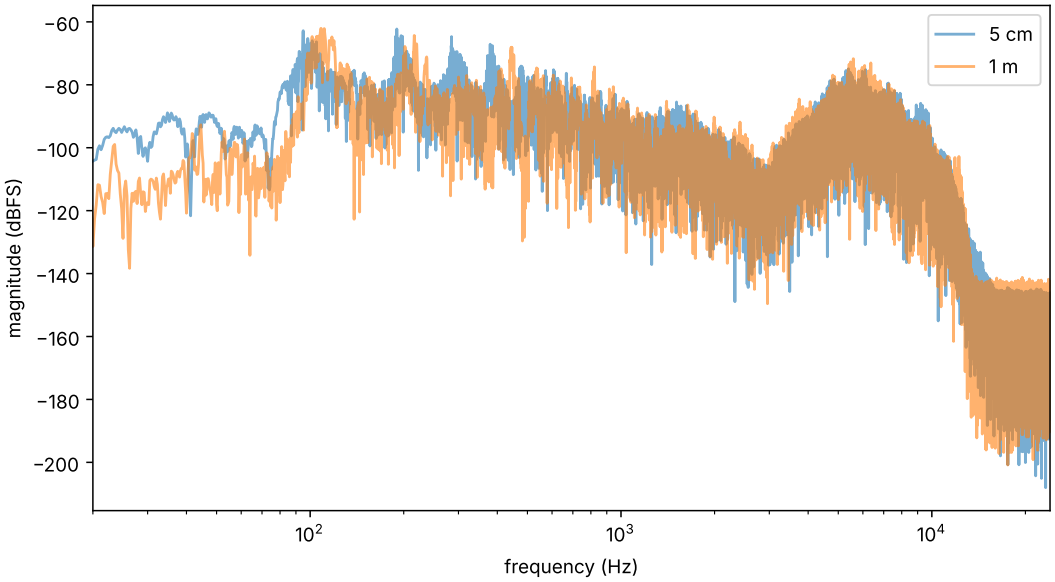


Figure 5: Enhanced spectra, 5 cm vs 1 m.

4 Task 3: Aural Exciter

4.1 Design overview

The exciter is built on the 5 cm recording and follows the Aphex block diagram (Figure 6). The dry signal passes straight through to the mixer. In the exciter sidechain a band of the signal is isolated by zeroing all FFT bins outside it (`band_pass`, built from two `gain_ctrl` calls with $-\infty$ dB, in place of a tunable high-pass filter), normalised to a peak of 1 so that the drive is always effective, and passed through $\tanh(\text{drive} \cdot x)$. A second band-pass then keeps only the wanted harmonics, which are scaled by the mix ratio and added to the dry signal.

tanh only starts to distort once its input reaches ± 1. For a sine input of sufficient drive the peaks are squashed into a square-like shape, which adds sine waves at multiples of the input frequency. Because tanh is odd, the top and bottom are squashed equally and only odd harmonics are produced. A sidechain centred on 2 kHz therefore places the third harmonic at 6 kHz, where the ear is most sensitive, and the fifth at 10 kHz, which is not too high. Only a band is fed to the non-linearity because the bandwidth of the output grows with the bandwidth of the input and a wide input produces muddy, unpleasant intermodulation. Only some harmonics are kept so that the generated content stays in the region where it is wanted (above 5.5 kHz) and the low harmonics are not added to the voice. The big-bottom sidechain of the Aphex diagram (low-pass 50 Hz to 190 Hz) is not implemented.

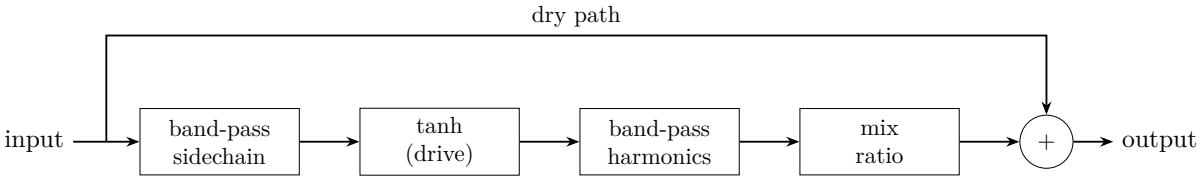


Figure 6: Block diagram of the implemented exciter.

4.2 Pre-processing

The exciter is run on the raw 5 cm recording. The pop at 1.28 s to 1.30 s lies mostly below 150 Hz (15 dB above the surrounding signal in the raw recording), outside the sidechain band, so it is not fed into the non-linearity and no separate pop removal is applied in this task.

4.3 Iterative development

Each iteration was judged by listening and by plotting the dry voice, the full tanh output and the added harmonics on one spectrum (Figures 7 and 8). Parameters are given as sidechain band, harmonic selection band, drive and mix. The dry recording has a peak of 0.0753 and an RMS of 0.0082. For each iteration the level of the added harmonics relative to the dry voice in 5.5 kHz to 7 kHz, and the peak and RMS of the output, are given in Table 3.

4.3.1 Iteration 1: initial design

Sidechain 1750 Hz to 2250 Hz (500 Hz wide, centred on $f = 2$ kHz), harmonics kept 5 kHz to 7 kHz, drive 10, mix 0.5. A band-pass was used instead of a high-pass because the narrower input reduces mud. The plot shows that tanh works as expected: only odd harmonics appear, at 6 kHz (3rd), 10 kHz (5th) and 14 kHz (7th). The result sounds robotic with a high-frequency metallic character. The added harmonics are 15.4 dB louder than the voice itself in 5.5 kHz to 7 kHz, and the output peak is 0.242, 3.2 times the dry peak, with the RMS tripled to 0.0252, so the drive and mix were reduced.

4.3.2 Iteration 2: lower drive and mix

Same bands, drive 3 and mix 0.1, so that less signal is pushed into tanh and the added harmonics are quieter. The added harmonic peaks sit at about -90 dBFS around 6 kHz. The harmonics are now 12.5 dB below the voice and the output peak is unchanged (0.0753). The boost in the desired region is clearly audible and the k and click sounds are more present, but the effect is limited to a single narrow region, so the next step widens the range of frequencies.

4.3.3 Iteration 3: wider bands

Sidechain 2000 Hz to 4000 Hz, harmonics kept 4 kHz to 12 kHz, drive 3, mix 0.25. The harmonics now cover a wider range, but the result shows more ringing than iteration 1; the output peak rises to 0.129 (1.7 times the dry peak) although the RMS stays at 0.0084.

4.3.4 Iteration 4: later harmonic band, higher drive

The harmonic selection band was moved to start at 5.5 kHz (keeping the third harmonic) and end at 12 kHz, with mix reduced to 0.15 to keep the output peak close to the level of the dry signal, and the drive was increased in steps up to 6. With drive 6 and mix 0.15 the harmonics are 8.2 dB below the voice and the output peak is 0.0985. The harmonics are clearly audible, but the consonants become too prominent and the voice develops a lisp.

4.3.5 Iteration 5: final design

The drive was reduced to 4, the harmonic selection band was narrowed to end at 10 kHz, and the mix was varied up and down until a sweet spot was found at 0.25. The k and s sounds are more pronounced and the voice sounds closer, while still sounding similar to the original. The harmonics are 10.2 dB below the voice in 5.5 kHz to 7 kHz, and the output has a peak of 0.116 and an RMS of 0.0084.

Table 3: Exciter output for each iteration (dry: peak 0.0753, RMS 0.0082).

It.	Sidechain (Hz)	Keep (Hz)	Drive	Mix	Harmonics vs dry (dB)	Peak	RMS
1	1750–2250	5000–7000	10	0.5	+15.4	0.242	0.0252
2	1750–2250	5000–7000	3	0.1	−12.5	0.075	0.0083
3	2000–4000	4000–12000	3	0.25	−15.4	0.129	0.0084
4	2000–4000	5500–12000	6	0.15	−8.2	0.099	0.0086
5 (final)	2000–4000	5500–10000	4	0.25	−10.2	0.116	0.0084

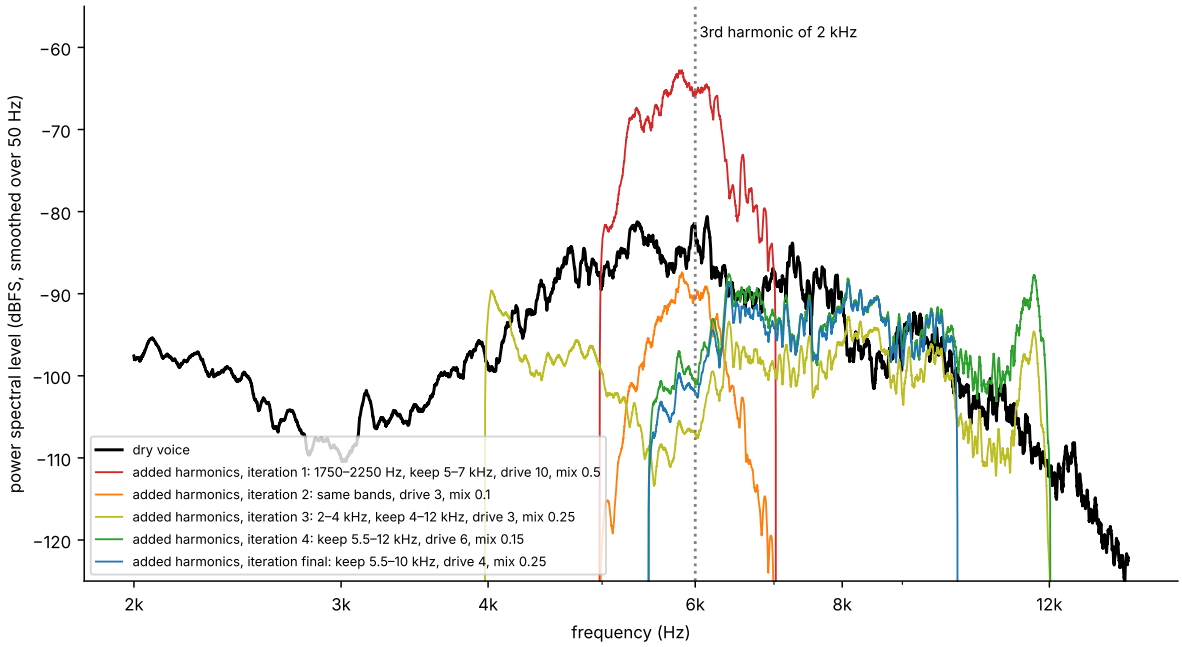


Figure 7: Dry voice and the harmonics added in each iteration, 2 kHz to 14 kHz (power level smoothed over 50 Hz). The dotted line marks 6 kHz, the third harmonic of the 2 kHz sidechain; the added harmonics of iteration 1 exceed the voice by 15 dB there.

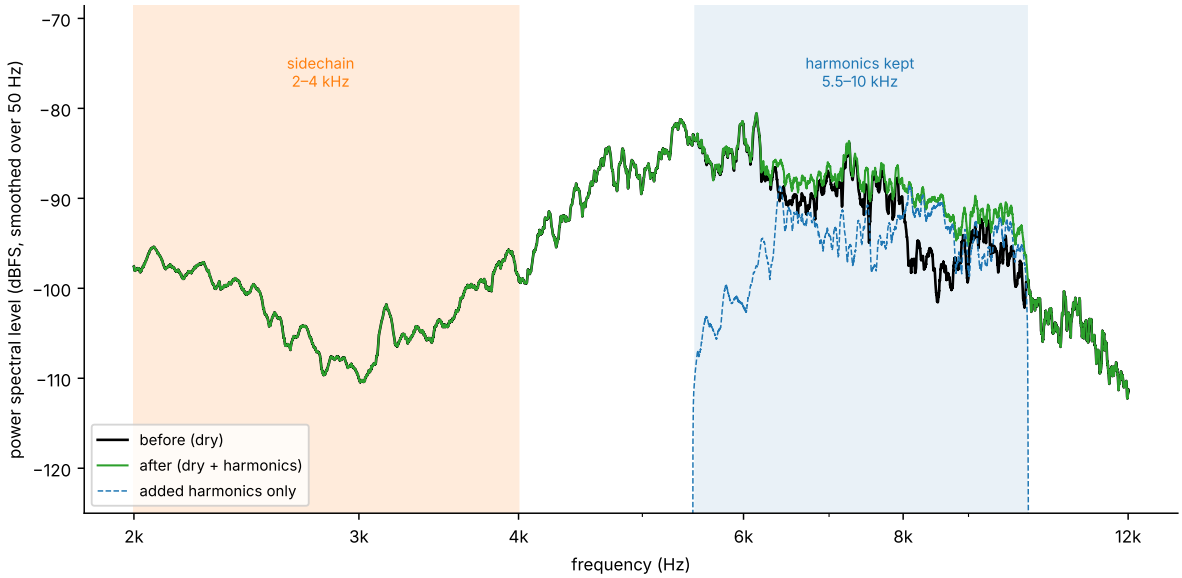


Figure 8: Annotated spectrum (2 kHz to 12 kHz) before and after the final exciter, showing the sidechain band (2 kHz to 4 kHz) and the harmonics kept (5.5 kHz to 10 kHz).

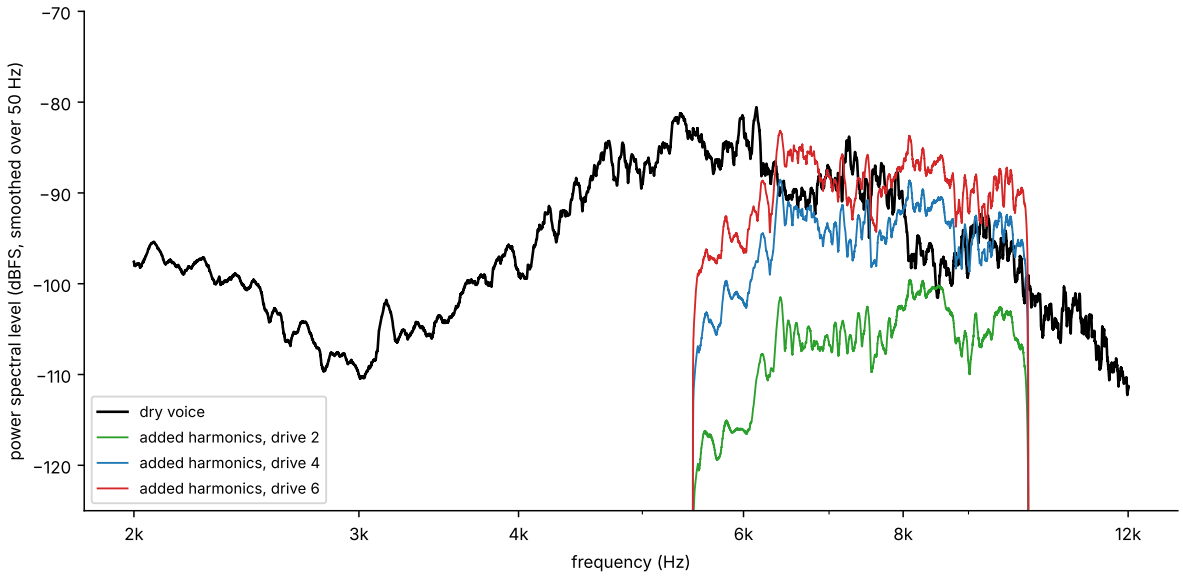


Figure 9: Effect of drive on the added harmonics with the final bands and mix 0.25: the harmonics are 23.8 dB, 10.2 dB and 3.8 dB below the voice at drive 2, 4 and 6 (5.5 kHz to 7 kHz).

4.4 Final design and justification

Table 4: Final exciter parameters.

Parameter	Value
Input recording	5 cm, raw
Sidechain band (Hz)	2000–4000
Sidechain normalisation	peak = 1
Non-linearity / drive	tanh, drive = 4
Harmonic selection band (Hz)	5500–10000
Mix ratio	0.25
Big-bottom band (Hz) (if used)	not used
Output peak / RMS	0.116 / 0.0084 (dry: 0.0753 / 0.0082)

The selection band starts at 5.5 kHz so that the third harmonic of the sidechain band is kept, and ends at 10 kHz, which removes the highest content that caused the lisp at drive 6. A drive of 4 and a mix of 0.25 were the settings at which the consonants are more pronounced and the voice sounds closer without sounding different or harsh. The sweep in Figure 9 shows why: at drive 2 the harmonics are 23.8 dB below the voice and the output peak (0.083) barely changes, at drive 6 they are only 3.8 dB below the voice, the peak rises to 0.132 and the RMS to 0.0090, and this is the setting that produced the lisp. The final design keeps the RMS within 0.2 dB of the dry voice (0.0084 against 0.0082) and the peak 3.7 dB higher, whereas iteration 1 raised the RMS by 9.7 dB and the peak by 10.1 dB. The added harmonics are all odd and lie in 5.5 kHz to 10 kHz, so the voice is made more prominent at the most sensitive frequencies without a noticeable increase in level.

A Acknowledgement of GenAI Use

Tools: Claude Opus 5.5 (Anthropic); Claude Sonnet 5.5 (Anthropic); Gemini 3.1 Pro (Google), plus various other Gemini versions for AI overviews (Google); GitHub Copilot (Microsoft).
Use: Claude Opus 5.5 was used to debug code syntax and logic and to suggest ways of making the code more efficient; to suggest the dBFS scale for the frequency-domain magnitude; to suggest an optimised pipeline for Task 2; to check the Task 1 readings against our own analysis; to explain the exciter and how a non-linearity creates harmonics; to fix a gain curve that rose at both ends of the spectrum; and to plot the overlay of both enhanced recordings and the harmonics for Task 3. Gemini 3.1 Pro was used for debugging, building LaTEX framework for the report, helping plot graphs, and checking the report. Gemini AI overviews for general lookups, and GitHub Copilot for inline autocomplete.
Own contribution: The recordings; the visual analysis of the spectra in Task 1; the choice of every gain setting in Task 2 and of every parameter in the iterative Task 3 design, judged by listening and by the spectrum plots; the code structure; and writing, checking and editing this report.

B Usage instructions

From a terminal, while in the project folder, run:

`python3 run_tasks.py`

The scripts need `numpy`, `scipy` and `matplotlib`. The plots of Tasks 1–3 are opened with `plt.show()`. The files `enhanced_5cm.wav`, `enhanced_1m.wav` and `exciter_5cm_5.wav` are written to the working directory. `run_tasks.py` imports `task1_plot.py`, `task2_enhance.py` and `task3_exciter.py`, which must be in the same folder.

C Python Code

C.1 run_tasks.py

```python
import os
import matplotlib.pyplot as plt
from scipy.io import wavfile
import numpy as np
# IFNDEF type statements not required in python, it wont try to import a file twice.

os.chdir(os.path.dirname(os.path.abspath(__file__))) # run from any folder: wav/ inputs and .wav outputs are relative to this file

# Assignmnet files
from task1_plot import plot_audio
from task2_enhance import gain_ctrl, enhancer, plot_gain_array
from task3_exciter import exciter, plot_harmonics, plot_exciter_spectrum

# --- Recommended: 48kHz | Signed 24-bit PCM | Mono (imperative) ---

# Load audio files and normalise (32 bit int)
sample_rate_5cm, audio_5cm = wavfile.read("wav/5cm.wav")
audio_5cm = audio_5cm / (2**31)  # NORMALISE (32bits to store 24bit audio, bitwise shift?)

sample_rate_1m, audio_1m = wavfile.read("wav/1m.wav")
audio_1m = audio_1m / (2**31)


# ------ TASK 1 ------
```

```
25
26  # Plot audio files
27  plot_audio("5cm", sample_rate_5cm, audio_5cm, 90,110,2000,12000,14000)
28  plot_audio("1m", sample_rate_1m, audio_1m, 90,110,2000,12000,14000)
29
30  # ------ TASK 2 ------
31
32  ##TRIMMM
```

```
# --- AUDIO OVERLAY ---
def plot_overlay(signals):
    # signals: list of (label, sample_rate, audio)
    plt.figure(figsize=(8, 5))

    for label, fs, audio in signals:
        n = len(audio)
        mag_fft = 20 * np.log10(np.abs((1/n) * np.fft.fft(audio)) + 1e-12)  # +1e-12 avoids log(0)
        freqs = np.arange(n // 2) * fs / n

        plt.plot(freqs, mag_fft[: n // 2], alpha=0.6, label=label)

    plt.title("Enhanced audio: 5cm vs 1m (frequency domain)")
    plt.xlabel("frequency (Hz)")
    plt.xscale("log")
    plt.ylabel("magnitude (dB)")
    plt.legend()

    plt.tight_layout()

plot_overlay([("5cm", sample_rate_5cm, enhanced_5cm),
              ("1m", sample_rate_1m, enhanced_1m)])
# --- END OF OVERLAY ---

# ------ TASK 3 ------

#Run on RAW and ENHANCED audio

#exciter(.., .., in_low, in_high, keep_lo, keep_hi, drive, mix)
#apply exciter + save
# --- ATTEMPT1: f = 2kHz (3F = 6kHz). WIDTH = 500Hz
#exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 1750, 2250, 5000, 7000, drive=10, mix=0.5)
# --- ATTEMPT2:
#exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 1750, 2250, 5000, 7000, drive=3, mix=0.1)
# --- ATTEMPT3:
#exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 2000, 4000, 4000, 12000, drive=3, mix=0.25)
# --- ATTEMPT4:
#exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 2000, 4000, 5500, 12000, drive= 6, mix=0.15)
# --- ATTEMPT5:
#exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 2000, 4000, 5500, 12000, drive= 6, mix=0.15)
# --- ATTEMPT5+:
exciter_5cm, nonlin_5cm, harmonics_5cm = exciter(audio_5cm, sample_rate_5cm, 2000, 4000, 5500, 10000, drive=4, mix=0.25)

plot_harmonics(audio_5cm, nonlin_5cm, harmonics_5cm, sample_rate_5cm)
plot_exciter_spectrum(audio_5cm, harmonics_5cm, sample_rate_5cm, 2000, 4000, 5500, 10000)
wavfile.write("exciter_5cm_5.wav", sample_rate_5cm, ((2**31-1)*exciter_5cm).astype(np.int32))
# Show audio plots
plt.show()
```

C.2 task1_plot.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

# Function to plot audio data in time and frequency domain
```

```
def plot_audio(name, sample_rate, audio, vowels_Start, vowels_end, consonants_start,
    consonant_end, noise_start, noise_end=0):
    plt.figure(figsize=(12, 5))

    # Time domain
    n = len(audio)
    time = np.linspace(0, n / sample_rate, num=n)  # array of times

    # Frequency domain
    audio_fft = (1/n) * fft(audio)

    # decibels relative to full scale (dBFS) - the loudest level possible in a digital system.
    mag_fft = 20 * np.log10(np.abs(audio_fft) + 1e-10) # Added small offset to avoid log10(0)
    freqs = (np.arange(n // 2) * sample_rate / n)  # frequency bins for plotting

    print(f"File: {name}, Sample Rate: {sample_rate}")

    # time domain plot
    plt.suptitle(f"Audio Analysis: {name}")
    plt.subplot(1, 2, 1)
    plt.title("time domain")
    plt.plot(time, audio)
    plt.xlabel("time (s)")
    plt.ylabel("amplitude")

    # frequency domain plot
    plt.subplot(1, 2, 2)
    plt.title("frequency domain")
    plt.plot(freqs, mag_fft[: n // 2])
    plt.xlabel("frequency (Hz)")
    plt.xscale("log")  # make logarithmic scale like standard EQ's/visualisers
    plt.ylabel("magnitude (dB)")

    plt.tight_layout()  # stop axes overlapping
    plt.show()

    # vowel/consonant/noise id

    # Calculate harmonics
    harmonic1_start = vowels_Start * 2
    harmonic1_end = vowels_end * 2
    harmonic2_start = vowels_Start * 3
    harmonic2_end = vowels_end * 3
    harmonic3_start = vowels_Start * 4
    harmonic3_end = vowels_end * 4

    # If noise_end wasn't provided, default it to the Nyquist frequency
    if noise_end == 0:
        noise_end = sample_rate / 2

    # Plot setup
    plt.figure(figsize=(12, 5))
    plt.suptitle(f"Annotated frequency domain: {name}")
    plt.plot(freqs, mag_fft[: n // 2])
    plt.xlabel("frequency (Hz)")
    plt.xscale("log")
    plt.ylabel("magnitude (dB)")

    # Get y-limits after plotting so labels sit at the very bottom/top
    y_bottom, y_top = plt.ylim()
    pad = 0.02 * (y_top - y_bottom)  # small gap from the edge

    # Annotate Vowels
```

```
    plt.axvspan(vowels_Start, vowels_end, color="red", alpha=0.25)
    plt.text(np.sqrt(vowels_Start * vowels_end), y_bottom + pad, "Vowels", color="red", rotation
    =90, horizontalalignment='center', verticalalignment='bottom')

    # Annotate Harmonics
    plt.text(np.sqrt(harmonic1_start * harmonic1_end), y_bottom + pad, "Harmonics 1, 2, 3",
    color="orange", rotation=90, horizontalalignment='center', verticalalignment='bottom')

    plt.axvspan(harmonic1_start, harmonic1_end, color="orange", alpha=0.25)

    plt.axvspan(harmonic2_start, harmonic2_end, color="orange", alpha=0.25)

    plt.axvspan(harmonic3_start, harmonic3_end, color="orange", alpha=0.25)

    # Annotate Consonants
    plt.axvspan(consonants_start, consonant_end, color="blue", alpha=0.25)
    plt.text(np.sqrt(consonants_start * consonant_end), y_bottom + pad, "Consonants", color="
    blue", rotation=90, horizontalalignment='center', verticalalignment='bottom')

    # Annotate Noise (top of the graph)
    plt.axvspan(noise_start, noise_end, color="gray", alpha=0.25)
    plt.text(np.sqrt(noise_start * noise_end), y_top - pad, "Noise", color="gray", rotation=90,
    horizontalalignment='center', verticalalignment='top')

    plt.tight_layout()
    plt.show()

    return None
```

C.3 task2_enhance.py

```
import numpy as np
import matplotlib.pyplot as plt #plot the envelope/EQ visualisation?
from scipy.fft import fft, ifft


# GAIN CONTROL
#function that describes gain adjustments to specific frequency ranges.
def gain_ctrl(low, high, amp_dB, sample_rate, gain_array, q=0):
    """
    q = 0    -> ramps one FFT bin wide (default) (rectangular)
    q = 0.25 -> each edge ramps over 25% of the bandwidth (trapazoidal)
    q = 1    -> ramps meet in the middle (triangular)
    """
    n = len(gain_array)
    half = n // 2
    bin_hz = sample_rate / n
    freqs = np.arange(half + 1) * bin_hz

    q = np.clip(q, 0.0, 1.0)                  # >1 would make the ramps cross
    ramp_hz = max(q * (high - low), bin_hz)   # never narrower than one bin
    r = ramp_hz / 2

    # bands that start at DC or end at Nyquist: push that ramp outside the spectrum so the ends
    get the full gain
    if low <= 0:
        low = -ramp_hz
    if high >= sample_rate // 2 - 1:
        high = sample_rate / 2 + ramp_hz

    xp = [low - r, low + r, high - r, high + r]
    fp = [0.0,     amp_dB,  amp_dB,   0.0]
    profile_dB = np.interp(freqs, xp, fp, left=0.0, right=0.0)
```

```
32
33      gain_array[:half + 1] *= 10.0 ** (profile_dB / 20.0)
34      gain_array[half + 1:] = gain_array[1:n - half][::-1]   # mirror to the negative frequencies
35      return gain_array
36
37  # PLOT GAIN
38  def plot_gain_array(gain_array, sample_rate, name):
39      n = len(gain_array)
40      freqs = (np.arange(n // 2) * sample_rate / n)  # frequency bins for plotting
41
42      plt.figure(figsize=(10, 5))
43      plt.title("Gain Control Curve " + name)
44      plt.plot(freqs, 20 * np.log10(np.abs(gain_array[:n // 2])), color='green')
45      plt.xscale("log")
46      plt.xlabel("Frequency (Hz)")
47      plt.ylabel("Gain (dB)")
48      plt.grid(which='both', linestyle='--', linewidth=0.5)
49      plt.tight_layout()  # plt.show() is called once at the end of run_tasks.py
50
51      return None
52
53  # --- ENHANCER ---
54  #takes gain array and applies it to audio
55  def enhancer(audio, gain_array):
56
57      #FFT the full signal (all (n) bins)
58      audio_fft = fft(audio)
59
60      #multiply the bins by gain_array
61      audio_enhanced = audio_fft * gain_array
62
63      #IFFT gives back all (n) samples, then keep the real part
64      audio_ifft = ifft(audio_enhanced)
65      audio_ifft_real = np.real(audio_ifft) #extract the real part (IM part should be ~0 since
        gain array is mirrored)
66
67      return audio_ifft_real
```

C.4 task3_exciter.py

```
1   import numpy as np
2   import matplotlib.pyplot as plt
3   from scipy.fft import fft
4
5   from task2_enhance import gain_ctrl, enhancer
6
7       #Dry path
8
9       #Exciter sidechain (HPF: 800-6kHz)
10
11      # Big bottom sidechain (LPF: 50-190Hz)
12
13      #Plot spectrum zoomed between 2-12kHz before and after each change (report: annotated graphs
        )
14
15  #Band selection (used for both the sidechain and the harmonic selection)
16  def band_pass(audio, sample_rate, lo, hi):
17      gain_array = np.ones(len(audio))
18      gain_ctrl(0, lo, -np.inf, sample_rate, gain_array) #low cut (0-lo)
19      gain_ctrl(hi, sample_rate//2 -1, -np.inf, sample_rate, gain_array) #high cut (hi+)
20      return enhancer(audio, gain_array)
21
22  def exciter(audio, sample_rate, in_lo, in_hi, keep_lo, keep_hi, drive, mix):
```

```
    #isolate the band
    sidechain_exciter = band_pass(audio, sample_rate, in_lo, in_hi)
    sidechain_exciter = sidechain_exciter / np.max(np.abs(sidechain_exciter)) #peak of 1, so drive is always effective

    #introduce non-linearity
    nonlin = np.tanh(drive * sidechain_exciter)

    #isolate the band around 6kHZ (5-7kHz? [TUNE]) - this rejetcs 5th harmonics since they lie outside the region
    target_band = band_pass(nonlin, sample_rate, keep_lo, keep_hi)

    #MIXER
    harmonics = mix * target_band
    return audio + harmonics, nonlin, harmonics #output, all harmonics from tanh, harmonics actually added

#Plot the harmonics on their own (for report)
def plot_harmonics(audio, nonlin, harmonics, sample_rate):
    n = len(audio)
    freqs = (np.arange(n // 2) * sample_rate / n)  # frequency bins for plotting

    plt.figure(figsize=(10, 5))
    plt.title("Exciter harmonics")
    for label, x in (("dry voice", audio), ("tanh output (all harmonics)", nonlin), ("added harmonics", harmonics)):
        mag_fft = 20 * np.log10(np.abs((1/n) * fft(x)[:n // 2]) + 1e-12)  # +1e-12 avoids log(0)
        plt.plot(freqs, mag_fft, alpha=0.6, label=label)
    plt.xscale("log")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("magnitude (dB)")
    plt.ylim(bottom=-180)
    plt.legend()
    plt.tight_layout()

#Plot the spectrum zoomed to 2-12kHz before and after the exciter (for report)
def plot_exciter_spectrum(audio, harmonics, sample_rate, in_lo, in_hi, keep_lo, keep_hi):
    n = len(audio)
    freqs = (np.arange(n // 2) * sample_rate / n)  # frequency bins for plotting
    zoom = (freqs >= 2000) & (freqs <= 12000)

    plt.figure(figsize=(12, 5))
    plt.title("Exciter spectrum (2kHz to 12kHz)")
    for label, x, style in (("before (dry)", audio, "k-"), ("after (dry + harmonics)", audio + harmonics, "g-"), ("added harmonics only", harmonics, "b--")):
        mag_fft = 20 * np.log10(np.abs((1/n) * fft(x)[:n // 2]) + 1e-12)  # +1e-12 avoids log(0)
        plt.plot(freqs[zoom], mag_fft[zoom], style, label=label)
    plt.xscale("log")
    plt.xticks([2000, 3000, 4000, 6000, 8000, 12000], ["2k", "3k", "4k", "6k", "8k", "12k"])
    plt.xlim(2000, 12000)
    plt.ylim(bottom=-150)
    plt.xlabel("frequency (Hz)")
    plt.ylabel("magnitude (dB)")

    y_bottom, y_top = plt.ylim()
    pad = 0.02 * (y_top - y_bottom)  # small gap from the edge

    # Annotate Sidechain
    plt.axvspan(in_lo, in_hi, color="orange", alpha=0.25)
    plt.text(np.sqrt(in_lo * in_hi), y_top - pad, f"sidechain\n{in_lo/1000:g}-{in_hi/1000:g} kHz", color="orange", horizontalalignment='center', verticalalignment='top')

    # Annotate Harmonics kept
```

```
    plt.axvspan(keep_lo, keep_hi, color="blue", alpha=0.25)
    plt.text(np.sqrt(keep_lo * keep_hi), y_top - pad, f"harmonics kept\n{keep_lo/1000:g}-{
    keep_hi/1000:g} kHz", color="blue", horizontalalignment='center', verticalalignment='top')

    plt.legend(loc="lower left")
    plt.tight_layout()

#ALSO PLOT COMPARISONS OF DIFFERENT DRIVE LEVELS
#TUNE THE SYSTEM AND RECORD IT TO SHOWCASE ITERATIVE DEVELOPMENT (REPORT)
```